

Total No. of Questions : 8]

SEAT No. :

PA-1451

[Total No. of Pages : 2

[5926]-67

T.E. (Computer Engineering)
ARTIFICIAL INTELLIGENCE
(2019 Pattern) (Semester - II) (310253)

Time : 2½ Hours]

[Max. Marks : 70

Instructions to the candidates:

- 1) *Attempt Q.1 or Q.2, Q.3 or Q.4, Q.5 or Q.6, Q.7 or Q.8.*
- 2) *Neat diagrams must be drawn wherever necessary.*
- 3) *Assume suitable data, if necessary.*

Q1) a) Explain Min Max and Alpha Beta pruning algorithm for adversarial search with example. [9]

b) Define and explain Constraints satisfaction problem. [9]

OR

Q2) a) Explain with example graph coloring problem. [9]

b) How AI technique is used to solve tic-tac-toe problem. [9]

Q3) a) Explain Wumpus world environment giving its PEAS description. [9]

b) Explain different inference rules in FOL with suitable example. [8]

OR

Q4) a) Write an propositional logic for the statement, [10]

i) "All birds fly"

ii) "Every man respect his parents"

b) Differentiate between propositional logic and First order logic. [7]

P.T.O.

- Q5)** a) Explain Forward chaining algorithm with the help of example. [9]
b) Write and explain the steps of knowledge engineering process. [9]

OR

- Q6)** a) Explain Backward chaining algorithm with the help of example [9]
b) Write a short note on : [9]
i) Resolution and
ii) Unification

- Q7)** a) Write a short note on planning agent, state goal and action representation. [6]
b) Explain different components of planning system. [6]
c) Explain the components of AI. [5]

OR

- Q8)** a) What are the types of planning? Explain in detail. [6]
b) Explain Classical Planning and its advantages with example. [6]
c) Write note on hierarchical task network planning. [5]



Q1)

a) **Min-Max and Alpha-Beta Pruning Algorithm for Adversarial Search:**

1. **Min-Max Algorithm:**

The Min-Max algorithm is used to make decisions in adversarial games (like chess, tic-tac-toe). It simulates the possible moves for both players, assuming the opponent plays optimally. The algorithm explores the game tree and assigns a score to the terminal states (leaf nodes), with the idea that one player is trying to maximize their score (Max), and the other is trying to minimize it (Min).

Steps:

- At each level of the tree, the algorithm alternates between the maximizing player (Max) and the minimizing player (Min).
- The algorithm recursively computes the value of each node in the tree until the terminal states are reached.
- The value at a leaf node is assigned based on the outcome of the game (win, loss, or draw).
- Once all terminal nodes are evaluated, the Min-Max algorithm chooses the optimal move by backtracking the values up the tree, with Max selecting the maximum value and Min selecting the minimum value.

****Example**:**

Let's say we are playing tic-tac-toe and have the following simplified game tree:

Max | Min | Max

(Player X moves first)

- ****Max**** (Player X) tries to maximize their score by choosing the best move that gives the highest value.
- ****Min**** (Player O) tries to minimize the score, forcing Max to choose a less optimal move.

After exploring all possible moves and terminal states, the Min-Max algorithm determines the best move for Player X, assuming both players play optimally.

2. ****Alpha-Beta Pruning****:

Alpha-Beta pruning is an optimization technique for the Min-Max algorithm that avoids exploring branches of the game tree that are not going to be chosen, thus saving computation time.

- ****Alpha****: The best score that the maximizer can guarantee so far (initially negative infinity).
- ****Beta****: The best score that the minimizer can guarantee so far (initially positive infinity).

- If at any point the algorithm encounters a node where the score is worse than a previously explored node, it "prunes" the branch (skips it).

****Steps**:**

- Start with the root node and initialize Alpha to negative infinity and Beta to positive infinity.
- Traverse the tree like Min-Max, but while evaluating nodes, prune branches where further exploration will not affect the final result.

****Example**:**

Imagine a game tree where Player X (Max) has already explored a move that guarantees a score of 5, and Player O (Min) has explored a move that guarantees a score of 2. If during Player O's exploration, a node is found with a score of 3, Alpha-Beta pruning will stop exploring that branch, as it is clear that Player O will choose a score of 2, and this branch will not change the outcome.

b) ****Constraints Satisfaction Problem (CSP)**:**

A ****Constraint Satisfaction Problem (CSP)**** is a mathematical problem defined by a set of variables, domains for each variable, and a set of constraints that the solution must satisfy. The goal

is to assign values to the variables such that all constraints are satisfied.

****Components of a CSP**:**

1. ****Variables****: These are the unknowns that need to be assigned values (e.g., in the N-Queens problem, each variable represents a queen's position on the board).
2. ****Domains****: These are the possible values each variable can take (e.g., in the N-Queens problem, each variable might have a domain of integers from 1 to N representing row positions).
3. ****Constraints****: These are the restrictions that the variables must satisfy (e.g., no two queens can share the same row or column).

****Example**:**

Consider the classic ****N-Queens problem****, where the goal is to place N queens on an $N \times N$ chessboard such that no two queens threaten each other. The CSP for this problem includes:

- ****Variables****: Each queen's position on the board.
- ****Domains****: The rows where each queen can be placed.
- ****Constraints****: No two queens can be in the same row, column, or diagonal.

****Solution Method**:**

The solution involves finding a set of assignments that satisfies

all constraints.

Q2)

a) **Graph Coloring Problem**:

Graph coloring is a problem in which we are given a graph, and the goal is to assign a color to each vertex such that no two adjacent vertices share the same color. This problem is commonly used to model scheduling problems, where adjacent tasks must not be scheduled at the same time.

Example:

Consider a graph with 4 vertices and edges between them:

- Vertices: A, B, C, D
- Edges: (A-B), (B-C), (C-D)

The task is to color the vertices such that no two adjacent vertices have the same color. A valid coloring would be:

- A: Red
- B: Blue
- C: Red
- D: Blue

In this case, adjacent vertices (e.g., A and B) have different

colors.

****Graph Coloring Algorithm**:**

1. Start by assigning a color to the first vertex.
2. For each subsequent vertex, assign the first available color that is not used by any of its adjacent vertices.
3. Continue until all vertices are colored.

****Applications**:**

- Scheduling: Assigning different times or resources to tasks that conflict.
- Map coloring: Assigning different colors to regions on a map so that adjacent regions do not share the same color.

b) ****AI Technique for Solving Tic-Tac-Toe Problem**:**

The ****Tic-Tac-Toe**** problem can be solved using various AI techniques, including ****Min-Max Search**** and ****Alpha-Beta Pruning****. These techniques help in making optimal decisions by simulating all possible moves in the game tree.

****Steps**:**

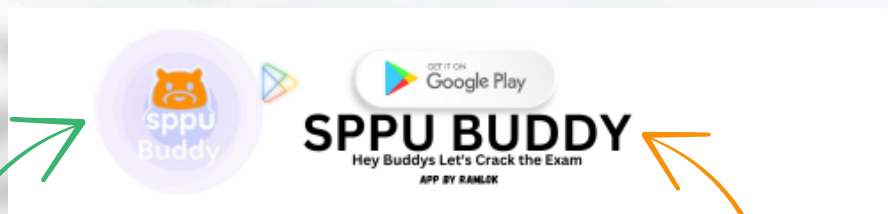
1. ****Game Representation****: The game is represented as a 3x3 grid, where each cell can be empty, occupied by "X", or occupied by "O".

2. ****Game Tree****: All possible moves from the current game state are explored recursively to create a game tree. The leaves of the tree represent terminal game states (win, loss, or draw).
3. ****Min-Max Search****: The Min-Max algorithm is used to evaluate the game tree. The algorithm assigns a value to each terminal state, where a win for "X" is +1, a loss for "X" is -1, and a draw is 0. The algorithm works by backtracking through the tree and choosing the move that maximizes the player's score (Max for "X", Min for "O").
4. ****Alpha-Beta Pruning****: To speed up the process, Alpha-Beta pruning can be used to eliminate branches of the game tree that will not affect the outcome.
5. ****Move Selection****: The AI (Player X or O) selects the optimal move by traversing the game tree and choosing the move that maximizes its chances of winning.

By using these AI techniques, the Tic-Tac-Toe game can be solved optimally, ensuring no player loses if both play optimally.



Download FULL PDF From APP
SPPU BUDDY

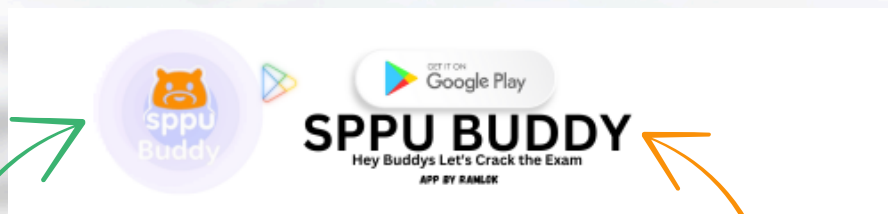


Download Now

ALL SUBJECT SOLVED PAPER'S ARE HERE



Download FULL PDF From APP
SPPU BUDDY



Download Now

ALL SUBJECT SOLVED PAPER'S ARE HERE